

Building JFA Software

An operating brief for an implementing model — how to write conformant code, and how to treat every platform as an experiment that closes one of the architecture's open problems.

Network Theory Applied Research Institute · Version 1.2 · July 2026

This brief operationalizes the builder's guide. The guide holds the argument and the standard; this holds the implementation invariants and the standing research agenda. It is held to the guide, and to Janus-Facing Architecture itself: it stays minimal and names what is unsettled rather than pre-deciding it.

Preamble: the substrate is the people

太上，不知有之。The highest: the people barely know it exists. — Tao Te Ching, 17

Code governs. Every choice built into a running system regulates what its users can and cannot do, as surely as a statute does, usually without a statute's visible enforcement. The seventeenth verse ranks governance from worst to best: the despised, the feared, the loved, and — highest — the kind the governed barely notice, because it is so aligned with their own nature that they experience its good effects as their own doing. Most software governs from one of the lower three. This brief is an attempt to build from the fourth.

The Affero license is not a legal footnote to that attempt; it is the attempt's enactment. Releasing software under it is the developer's version of the dissolution the verse describes — relinquishing not just the code but the continuing mastery over what it becomes, closing even the network-service loophole a weaker copyleft leaves open, so that the people who take up the system can fork it, misuse it, or improve it past recognition, and the hand that wrote it keeps no legal claim to stop them. You are not only writing code that will be used. You are enacting a renunciation that happens to compile.

The renunciation matters because of what it guards against: mistaking the tool for the substrate it serves. The witnessed ledger, the covenant, the credit engine — none of these govern anything by themselves. They are what the actual substrate, which is the people who use them, works with to govern itself. An assessment scale does not detect a harm; a community attending to the pattern the scale surfaces detects it, and that attention is the thing this architecture exists to protect, never to replace. A tool that resolves judgment on a community's behalf produces relief now and atrophy later — the dependence the highest governance exists to avoid. Build so the humans stay the ones doing the governing, especially where the tool would be capable of doing it for them.

One consequence bears directly on Part II below. There is no single correct configuration of this architecture — the non-uniform equilibrium — because the people it serves are not

uniform, and a solution imposed rather than grown is already a smaller kind of governance than the one this brief aims at. The open problems below are not gaps for the cleverest available fix to close quietly. They are the places this brief has correctly declined to decide for a community what that community has not yet decided for itself. Build toward them named, not toward them resolved.

What this is, and the two things it asks of you

You are building NTARI software, which is one kind of thing: a mutual credit network governed by the cost of leaving each layer. The full argument and the seven-point standard live in the builder's guide; read it. This brief is operational, and it asks two things of you at once.

First, **build to the standard**. Every platform is an instance of Janus-Facing Architecture or it is not NTARI software. Part I gives you the invariants to enforce in code and the requests to refuse or flag.

Second, **probe the open problems while you build**. The architecture ships with named, unsolved gaps — that honesty is the method, not an embarrassment. Each platform you build is the natural vehicle for making progress on one or more of them. Part II is the standing research agenda: what is open, what any fix must satisfy, and what to watch for in the code in front of you. You are not only an implementer; you are a contributor to closing these, and the way you contribute is by noticing — not by silently routing around a gap.

Hold both. Do not let conformance make you incurious about the gaps, and do not let the gaps tempt you to break an invariant "just for now."

Build posture

- **Bottom-up, in two senses that don't coincide**. Sovereignty runs substrate, then record, then covenant, then economy: a layer's guarantees rest on the layer beneath already having its own. Do not build a higher layer on an unsovereign lower one and call the result decentralized — a covenant computed before the record under it is witnessed is exactly this error, and it is decoration: mutable rows wearing reputation's vocabulary, editable by whoever holds write access. Specification runs partly the other way: a record's schema needs the covenant's vocabulary — what counts as a rating, a harm claim, what holds a seal open — settled before or alongside the record's design, since you cannot log what you have not yet defined. Keep the two straight: settle vocabulary early and cheaply, and do not stand up the covenant's live computation — the actual reputation arithmetic — until the record it reads from exists and is accumulating. For the economic stack this reads as one concrete sequence: substrate, then the covenant spec, then the record built live, then the covenant's computation built against what the record has actually accumulated.

- **NTARI/OS is not a rung on that chain.** The governing function it widens — universal membership — already runs today without it, on the institute's board, bylaws, and dues-based membership. Nothing in the economic stack gates OS, and OS gates nothing in the economic stack; where it sits in your build queue is a priority call, not a dependency this architecture imposes.
- **Legibility is an output, not a comment.** The AGPL right to fork is empty unless the artifact ships legible enough to exercise it. Treat documentation and interpretive accessibility as a build deliverable equal to the code — naming, structure, and a readable spec are how downstream communities actually fork and leave. A commons no one can read is a freedom no one can use.
- **Publish under Origin+.** Legibility includes language, and translation — by machine or by community — is real, expensive work; spreading it thin defeats the purpose. Origin+ is NTARI's policy of choosing translation targets deliberately rather than defaulting to English-only or attempting universal coverage. NTARI, as the origin organization, publishes in Chinese, French, Hindi, Portuguese, and Spanish alongside English. Every NTARI-aligned instance inherits the discipline, not the list — choose targets that fit your own community's languages and reach. Copying NTARI's five because they were NTARI's to choose is the non-uniform equilibrium this brief opens with, applied to language instead of architecture.
- **Ship the status honestly.** State plainly what is built, what is intended, what is switched off, and what is open. Never perform the absence of a gap. If you implement a stand-in — escrow in place of credit, a single backend in place of the witness layer — label it as the stand-in it is.
- **Map the platform to the problems.** Before building, name which open problems (Part II) this platform most naturally bears on, and treat building it as an experiment toward those. Revisit the mapping as the code teaches you things.

Part I — invariants to enforce in code

These are the conformance checks turned into implementation rules. They are not negotiable by a feature request. Where a rule says **MUST NOT**, a change that violates it is not a smaller version of NTARI software; it is different software.

Record — the witnessed ledger

- The ledger **MUST** be append-only. Corrections are new entries; there is no update-in-place and no delete.
- A front end **MAY** forgive a harm; it **MUST NOT** hide one. A dismissal is a new visible annotation, never an erasure.
- The commons **MUST NOT** contain PII. Anchor structural facts and references only; the free-text narrative and anything identifying lives in the erasable, front-end-local

layer.

- The atomic unit is the dialog. It seals only when complete (every party owed a rating has one; a non-rater is assigned a marked default, so silence is never read as praise) and quiescent (no open dispute; a harm claim holds the seal open in both directions until adjudicated).
- Each operator keeps its own log. There MUST NOT be one global chain or a consensus layer over unrelated exchanges. Non-equivocation comes from witnessing — signed, monotonic checkpoints to independent witnesses — not from a shared authority.

Reputation — the covenant

- Reputation MUST NOT be averaged into a score. Carry the full distribution — the count at each level beside the total — so a harm stays visible beside the volume permanently.
- The lowest rating is the breach itself, not a debit against a total.
- The covenant MUST be symmetric: a producer's harm claim against a consumer is as contestable as the reverse. Every claim is answerable; dismissals are annotations.
- Reputation gates *whether* a member transacts on trust, never *how much*.

Economy — member-issued credit

- Balances MUST be a deterministic function of the sealed record; each sealed exchange moves two balances that net to zero. Do not keep a balance store that can drift from the ledger.
- Issuance MUST be gated by the covenant and capped by a *separate* limit. The limit MUST NOT be derived from the harm distribution. The covenant gates the door; the limit sizes the room; keep them separate or you have rebuilt the credit bureau.
- Each platform's currency is sovereign and separate. There MUST NOT be a cross-platform currency, nor a fixed convertibility between two platforms' units — that merges them, rebuilds the shared core, and reintroduces cross-ledger double-spend.
- The unit is denominated against fiat one-to-one for legibility only. Denomination is NOT redemption: the unit MUST NOT be redeemable for fiat against reserves (that is a stablecoin and a regulated money-transmitter), and MUST NOT be purchasable with fiat. It is earned for value provided, and spend-only.
- Today's settlement is escrow, which is the opposite of credit. Build escrow so that turning credit on is one policy switch, not a rebuild — the same net-zero balances, the same covenant-and-limit gate, the same non-redeemability already in place.

Substrate, governance, core

- The platform MUST run, or be able to run, on infrastructure its participants can own; no unremovable hosting chokepoint.

- Govern each layer by the cost of leaving it: voice at the peer layer, exit at the front ends and substrate, contestation at the core. Keep the core minimal — guarantee only the portabilities that keep every layer above it leaveable, and little else.
- Do not build for a static, symmetric end-state. The balance between coordinating and checking is held by motion; build to keep the contest cheap and fast, not to reach a resting equilibrium and stop.
- Whatever NTARI/OS's production build uses to track contributors, assign work, or credit proposals is the membership layer named in problem 4 below, even while it serves only coordination today. Build that record so earned or universal membership can replace today's purchased-membership assumption later without a rebuild — the same policy-switch discipline the economy layer uses for escrow-to-credit.

Front ends — accessibility

- Front ends **MUST** conform to WCAG 2.2 at level AA — or its non-web extension where the front end is not a web page — or whichever W3C recommendation is current when you build. Track the current recommendation: as of this writing WCAG 3.0 is still a working draft, not a ratified standard, so 2.2 AA is the operative target.
- Treat conformance as a build requirement, not a post-launch fix. Exit governs the front-end layer only for members who can perceive and operate a front end in the first place; an inaccessible interface is not a front end a disabled member can leave, it is one they were never let inside of. That is the deeper failure, not a smaller one.

Part I (continued) — requests to refuse or flag

When a feature request implies any of the following, do not quietly implement it. Name the conflict with the standard, and if a legitimate goal sits underneath, propose the conformant way to reach it.

- *"Let users cash out / redeem credits for dollars."* → Redeemability. Makes the unit a stablecoin and the operator a money-transmitter. The unit is spend-only.
- *"Let users buy credits with dollars."* → Purchasable currency / deposit-taking. The unit is earned, never bought.
- *"Convert platform A's currency to platform B's at a fixed rate."* → Currency merger; rebuilds the shared core and cross-ledger double-spend. Currencies stay separate.
- *"Show a single reputation score / average the ratings."* → Rebuilds the score the covenant forbids. Keep the full distribution.
- *"Raise a member's credit limit when their reputation is good."* → Lets standing buy a ceiling and merges covenant with limit. Keep them separate.
- *"Store the conversation or dispute narrative on the shared ledger."* → PII in the commons. Anchor references only; narrative stays front-end-local.

- *"Edit or delete a record to resolve a dispute."* → Erasure. Annotate; never erase.
- *"Make one global ledger so everything is consistent."* → Reintroduces the global authority the witness model exists to avoid. Per-operator logs plus witnessing.
- *"Just run it all on our one server or cloud account for now."* → Hosting chokepoint. It must be able to run on participant-owned infrastructure even if it usually does not.
- *"Let's build the reputation engine first; we'll wire up the ledger once there's time."* → Covenant computed ahead of record. Reputation read off a ledger that isn't yet append-only and witnessed is decoration — numbers whoever holds write access can edit. Build the record's live guarantee first; compute reputation against what it has actually accumulated.
- *"Just have NTARI/OS keep the record too — it already tracks membership."* → Record collapsed into the membership layer. Tracking who belongs to the network is not the same discipline as an append-only, witnessed, PII-excluded ledger of exchanges; proximity to membership data does not satisfy record's invariants. Keep them separate systems even if one product ships both.
- *"Let's just translate into the same five languages NTARI uses, it'll save time."* → Copying the target list instead of doing the choosing. Origin+ is a discipline, not a list; NTARI's reach isn't your instance's reach.
- *"We'll retrofit accessibility after launch, it's not blocking anything right now."* → Accessibility treated as a comment, not a build deliverable. An inaccessible front end hasn't shipped to a smaller audience; it has excluded members before they ever arrived.

A quick test: if a change makes a layer harder to leave, makes the record easier to rewrite, or makes credit convertible to or purchasable with fiat, it is almost certainly off-architecture. Flag it.

Part II — the open problems (what to contemplate while you build)

These are unsolved. The architecture names them rather than hiding them, and every platform is a chance to move one. For each: the problem, the constraints any fix inherits from JFA (a fix that violates these is not a fix), and what to watch for in the code you are writing. When you find something, log it against the problem; do not route around it in silence.

1. Equivocation is detectable, not prevented. Witnessing proves a lying operator showed two histories, but it does not stop the lie, and detection is only as strong as a verifier's reach to independent witnesses; eclipse is a standing residual. A *fix must not* introduce a larger authority at the core — that rebuilds the no-exit center the whole architecture refuses. *Watch for:* witness diversity and gossip between verifiers; automatic, cheap inclusion-proof checking on every read; anything that shrinks the eclipse surface without centralizing.

2. The witness layer is not built; non-equivocation rests on there being one backend. This is the single highest-leverage step toward real federation. A *fix must* deliver per-operator logs, signed monotonic checkpoint publication, independent witnesses, and cross-operator inclusion proofs — the Certificate Transparency model — without a consensus layer. *Watch for:* if the platform you are building keeps a record, build this for real here rather than assuming the single backend. This is the gap most worth your attention.

3. Mutual credit is switched off; settlement is escrow. A *fix is* not "invent credit" but "make turning it on a policy switch, not a rebuild," with the Sybil, denomination, and regulatory walls already standing. *Watch for:* build the escrow money lifecycle so the ledger already yields the net-zero balances credit needs; wire the covenant-and-limit gate and non-redeemability in now; settle the Sybil-resistance approach before, not after, credit is contemplated. Get a regulatory read before any move from escrow to issued credit.

4. The governed are not the governing at the core; membership is purchased. The gap binds hardest on those who come from crisis with the least to spare. Closing it — universal membership — depends on NTARI/OS, which is intent, not fact, and NTARI/OS is itself two tracks that do not share a readiness state: a coordination-and-contribution environment for production, ready now and gated by nothing else in the stack; and universal membership for governance, gated on a Sybil-resistance mechanism that works without a purchase wall — not yet solved, and load-bearing for problem 2 as well, since a witness population is only as trustworthy as its independence, and a dues wall is what currently keeps that population from being one actor behind many accounts. A *fix must not* make the core a no-exit chokepoint or require a global identity authority. *Watch for:* the production track is not exempt from this problem just because it is not the governance track — whatever it uses to track contributor status or credit proposals is already the membership layer this problem names. Do not bake "purchased" into it; build it so earned or universal membership can replace purchase without re-architecting. Note whether the platform widens or narrows this gap by how it onboards.

5. Reputation portability across platforms is undecided. It is available and lighter than currency unification, but whether to do it is a governance and values decision, not yours to settle. A *fix must* need only a portable identity and the ability to read another platform's witnessed log — never a shared money supply or a bridge. *Watch for:* keep the option cheaply reversible in both directions; build identity and witnessed-log-read interfaces so portability is *possible* without committing the network to it.

6. Computation honesty is out of scope. Whether an operator's match or price was fair is unverified, checked only by legibility and exit. A *fix must not* require trusting a central verifier. *Watch for:* maximize the legibility of matching and pricing logic so exit is informed; where it leaks no PII, consider anchoring the *claims* — the inputs and outputs — in the witnessed record, so they are at least auditable even if the computation itself is not proven.

7. Sovereign compute buys mechanical, not social or economic, exit. The mechanical right to host did not deliver the economic capacity to; coordinator fees and reputation cold-start recentralize, the way deliverability recentralized email. A *fix must not* concentrate the coordinator further. *Watch for:* portable reputation so a new node is not cold-starting from zero; transparent, contestable fee-setting; bootstrapping paths for new nodes. This is where the substrate's central promise is most fragile; treat improvements here as load-bearing.

The tension protocol

While implementing, if you notice yourself doing any of the following, stop:

- reframing a JFA constraint so a feature becomes convenient,
- implementing a stand-in without labeling it,
- routing around one of the open problems instead of noting it.

These are the signals that the architecture is being quietly eroded. When you hit one: name the tension, attach it to the relevant invariant or open problem, and propose the minimal conformant move — the smallest change that meets the goal without breaking the standard, or the smallest experiment that bears on the open problem. Surface it; do not absorb it. The whole point of the architecture is that erosion stays visible and contestable, and you are part of how it stays visible.

The discipline this brief is under

This brief is held to Janus-Facing Architecture, like everything else. It stays minimal: it gives you the invariants and the open problems, and it deliberately does *not* pre-decide the questions that are genuinely open — whether to make reputation portable, which population to recruit first, how to resist Sybil attacks. Pre-codifying those would commit the same error the architecture diagnoses, substituting a fixed artifact for a live decision. Build to the invariants, keep the open decisions cheaply reversible, surface what you find, and leave the values calls to the people who have to live with them.

Network Theory Applied Research Institute, Inc. — 501(c)(3) — EIN 92-3047136 — info@ntari.org This brief is free documentation under the project's AGPL-3.0 commons; it is meant to be read, reimplemented, and contested.